
UNIT 8 Structured Query Language (Part-II)

Structure

- 8.0 Introduction
- 8.1 Objectives
- 8.2 SQL Joins
 - 8.2.1 Equi-join
 - 8.2.2 Outer Join
 - 8.2.3 Self-Join
- 8.3 Nested Queries
 - 8.3.1 Subqueries
 - 8.3.2 Correlated Subqueries
- 8.4 Database Objects
 - 8.4.1 Views
 - 8.4.2 Sequences
 - 8.4.3 Indexes and Synonyms
- 8.5 Summary
- 8.6 Solutions/Answers

8.0 INTRODUCTION

In the previous unit, you have gone through the details of the basic set of commands of the Structured Query Language (SQL). A good database system consists of normalised relations, which involve the lossless decomposition of relations into smaller relations. These smaller relations have controlled redundancy and help in removing the redundancy-related problems of a database, which you have studied in the earlier units of this course. However, when you want to retrieve information from a database, you may be required to extract information from several smaller relations. Join is one of the most important operations, which can be used to provide information from multiple tables. This unit discusses different types of join operations that can be performed on database tables.

In many database queries, you may be required to use a sub-query that provides data for the main query. This unit discusses the subqueries and correlated subqueries. The unit also introduces you to the concept of views, sequences, indexes and synonyms database objects. You must practice these queries, to master them.

8.1 OBJECTIVES

After going through this unit, you should be able to:

- Define different kinds of joins that can be performed on tables;
- Write the SQL statements for Join commands;
- Use subqueries and correlated subqueries;
- Write nested subqueries;
- Define and write SQL commands to create views and indexes;
- Define and write SQL commands for creating sequences and synonyms.

8.2 SQL JOINS

The join is an important relational operation for queries that involve more than one relation. You have already gone through the relational join operation in an earlier unit, this unit explains the use of the join operation in SQL. The purpose of join operation is to combine the data from two different tables on specific attribute(s), which share the same domain. *The necessary condition for the meaningful join of two tables is that each of these tables should have at least one attribute, called the joining attribute, that should have the same data domain.* The join operation uses the joining condition to combine the data of the two tables. In this unit, we will use the tables and data given in Figure 1 for various examples of join operations.

Table Name: CLIENT			
<u>ClientID</u>	ClientName	ClientAddress	ClientPhone
C001	ABCD	79, MGRoad	9999999991
C002	BCDE	Raman Street	9999999992
C003	CDEF	Vindyachal apatments	9999999993

Table Name: ITEM			
<u>ItemID</u>	ItemName	ItemPrice	QuantityAvailable
I01	Pen	20	500
I02	Pencil	5	1000
I03	Paper Sheet	20	1000
I04	Sharpener	5	200

Table Name: ORDER	
<u>OrderID</u>	ClientID
O001	C001
O002	C002
O003	C001
O004	C003
O005	C002

Table Name: ORDERDETAILS		
<u>OrderID</u>	<u>ItemID</u>	QuantityPurchased
O001	I02	2
O001	I03	10
O001	I04	1
O002	I02	5
O003	I01	2
O003	I03	5
O004	I01	5
O004	I03	3
O005	I01	8

Figure 1: Sample Tables and Data

The tables above do not have a date of purchase, as these tables are just for demonstrating various commands. Please note that the primary key of each table has been underlined. Please also note that each OrderID is unique, and an order may contain several items. The foreign keys in the tables are:

- OrderID in ORDERDETAILS table references ORDER table
- ItemID in ORDERDETAILS table references ITEM table
- ClientID in ORDER table references CLIENT table

Now, let us answer the following questions about the join operation:

1. Can you join the CLIENT and the ITEM tables? No, as there is no common attribute in these two tables on which table can be joined. Please note that two tables can be joined only if they have at least one attribute each which has the same domain.
2. How will you find the name of items that have been ordered by a client, whose name is ABCD? ClientID is C001? In order to answer it first let us assume that your DBMS supports the clause
 SELECT * INTO # <Name of a Temporary Table>
 FROM ...;

Now, you can find the relevant information in the following way (a very cumbersome way):

- a) First, find the ClientID of the client whose name is “ABCD” from the CLIENT table using the command:

```
SELECT ClientNo INTO #TempCLIENT
FROM CLIENT
WHERE ClientName = “ABCD”;
```

The output of this query would be a Temporary table named #TempCLIENT. As per the data of Figure 1, the only ClientID with the name ABCD is C001. Thus, the output of the command above would be:

Table Name: #TempCLIENT
<u>ClientID</u>
C001

- b) Next, you can find the orders made by ClientID C001 from the ORDER table using the command:

```
SELECT OrderID INTO #TempORDER
FROM ORDER
WHERE ClientID IN (      SELECT *
                        FROM #TempCLIENT);
```

The output of the command above would be:

Table Name: #TempORDER
<u>OrderID</u>
O001
O003

- c) Next, you will find the items that have been ordered in each of the OrderID O001 and O003 from the ORDERDETAILS table using the command:

```
SELECT DISTINCT ItemID INTO #TempORDERDETAILS
FROM ORDERDETAILS
WHERE OrderID IN (      SELECT *
                        FROM #TempORDER);
```

The output of the command above would be:

Table Name: #TempORDERDETAILS
<u>ItemID</u>
I02
I03
I04
I01

- d) Now, you can find the name of the ordered items by C001, from the ITEM table using the following command:

```
SELECT ItemID, ItemName
FROM ITEM
WHERE ItemID IN (      SELECT *
                     FROM #TempORDERDETAILS);
```

The output of the command above would be:

ItemID	ItemName
I02	Pencil
I03	Paper Sheet
I04	Sharpener
I01	Pen

Figure 2: The Output of the Command

Can this query be answered with a single query? You may require the JOIN command to solve this query.

```
SELECT DISTINCT i.ItemID, i.ItemName
FROM CLIENT c, ORDER o, ORDERDETAILS oo, ITEM i
WHERE c.ClientName = "ABCD" AND
      c.ClientID = o.ClientID AND
      o.OrderID = oo.OrderID AND
      oo.ItemID = i.ItemID
```

Though the statement above looks very complicated, you can interpret it by the following statements:

- In the select statement, you have put the information that you want to output. The ItemName information is available in only ITEM table, whereas ItemID is available both in ITEMDETAILS and ITEM tables, thus, the alias i before the ItemID is necessary and informing the database server that ItemID data is to be obtained from the ITEM table (please note i is an alias to ITEM table.).
- In the FROM clause have used aliases for every table that is being used.
- The WHERE clause has the following conditions:
 - c.ClientName = "ABCD", which will identify the ClientID as C001.
 - c.ClientID = o.ClientID will join the CLIENT table (alias c) with Order table (alias o), and output O001 and O003.
 - o.OrderID = oo.OrderID will join the ORDER and ORDERDETAILS tables
 - oo.ItemID = i.ItemID will join the ORDERDETAILS and ITEM tables.

Thus, producing the final result, as shown in Figure 2.

This example may seem to be very complex, but you must once again go through the example after going through the remaining part of this unit. Now, let us now focus on different kinds of joins.

8.2.1 Equi-join

Equi-join, as the name suggests, has equality as the joining condition. This means that the attributes that you are using to join would be checked for equality. Please note that the names of the joining columns in the two tables may be different. The following is an example of an equijoin operation:

Example 1: Using the tables of Figure 1 answer the query:

List the ID, name of the client, and the order ids of all the clients who have placed an order.

You need to use the CLIENT and ORDER tables to answer this query, as the name of the clients are in the CLIENT table and the order id information is in the ORDER table. You may use equi-join operation for this query using the columns ClientNo in CLIENT and ClientNo in the ORDER table. Though in this present example, both the columns have the same name, i.e. ClientID, equi-join operation does not require these names to be the same. SQL query for the query is presented below:

Query Using equi-join:

```
SELECT c.ClientID, c.ClientName, o.ClientID, o.OrderID
FROM CLIENT c, ORDER o
WHERE c.ClientID = o.ClientID;
```

The result of the query, on the tables of Figure 1, will be:

<u>c.ClientID</u>	ClientName	<u>o.ClientID</u>	OrderID
C001	ABCD	C001	O001
C001	ABCD	C001	O003
C002	BCDE	C002	O002
C002	BCDE	C002	O005
C003	CDEF	C003	O004

The output of shows the ClientID of both the CLIENT and the ORDER tables. We can display any one of these two columns. Please also note that joining may result in the display of redundant information, as you can observe in the Client name column.

Natural Join: Natural join is one of the most used join operations. It requires that the joining columns of the two tables should have the same name. It uses the equality condition. The result of the join operation displays only one of the joining attributes/columns.

The query of example 1 can also be answered using the natural join operation, as both the tables have the same column name ClientID. The following SQL command will perform the natural join operation:

```
SELECT CLIENT.ClientID, ClientName, OrderID
FROM CLIENT NATURAL JOIN ORDER;
```

The output of the command would be:

<u>CLIENT.ClientID</u>	ClientName	OrderID
C001	ABCD	O001
C001	ABCD	O003
C002	BCDE	O002
C002	BCDE	O005
C003	CDEF	O004

Inner Join: Inner join combines two tables using a combining condition on the joining attributes. In the present versions of SQL, you may have to use INNER JOIN command for example 1 query:

```
SELECT CLIENT.ClientID, ClientName, OrderID
FROM CLIENT INNER JOIN ORDER
ON CLIENT.ClientID = ORDER.ClientID;
```

8.2.2 Outer Join

Consider the situation that a new client has been added to the client table and the present state of the CLIENT table is:

Table Name: CLIENT			
<u>ClientID</u>	ClientName	ClientAddress	ClientPhone
C001	ABCD	79, MGRoad	9999999991
C002	BCDE	Raman Street	9999999992
C003	CDEF	Vindyachal apatments	9999999993
C004	DEFG	Maidan Road	9999999994

Further, assume that this new client has not issued any order. Let us now issue the query of example 1 again on the present state of the database. You will find the result of the query will still be the same, as shown in example 1. So, we get no information about C004, in the result of the query.

If the objective of the query was to show the list of all the clients and their OrderIDs, irrespective of the fact, that they have given zero or more orders, then how would you extract such information? In effect, you want that all the records from the CLIENT table should participate in joining even if there is no joining row in the ORDER table. This requires the use of OUTER JOIN operation.

Example 2: List all the clients and their orders (NULL in case no order is given by a client).

The following SQL command will be required for the query asked above:

```
SELECT CLIENT.ClientID, ClientName, OrderID
FROM CLIENT LEFT OUTER JOIN ORDER
ON CLIENT.ClientID = ORDER.ClientID;
```

We have used the term LEFT OUTER JOIN, as in this command we want that all the records of the table mentioned on the left side of the join command (CLIENT in this case) be part of the output, irrespective of a joining record on the table on the right side (ORDER in this case). The output of the command would be:

<u>CLIENT.ClientID</u>	ClientName	OrderID
C001	ABCD	O001
C001	ABCD	O003
C002	BCDE	O002
C002	BCDE	O005
C003	CDEF	O004
C004	DEFG	NULL

You may notice that the last record in this table is for Client C004, who has not placed any order yet. This record is displayed as we have used the Left Outer join operation. Similarly, you can use the RIGHT OUTER JOIN or FULL OUTER JOIN, as per the need of database output.

8.2.3 Self-Join

In the self-join operation, a table is joined with a copy of itself. This is very useful for queries, which draw information from within a column of a table. The following is an example of the self-join.

Example 3: Find the pairs of similar priced items in the ITEM table.

You may first simply inspect the ITEM table. You will find that pairs -Pen and Paper Sheet; and Pencil and Sharpener, have the same price. How did you find this information? You compared the ItemPrice of each item with other ItemPrice. This inspection, in general, can be performed by join operation. Hence, you can answer the query using the following SQL command:

```
SELECT i1.ItemName, i2.ItemName
```

```
FROM ITEM i1, ITEM i2
WHERE i1.ItemPrice = i2.ItemPrice;
```

Thus, you are joining the two tables on identical ItemPrice, and displaying the pair of names of the items that have similar prices. However, you will get the following output of this SQL command:

i1.ItemName	i2.ItemName
Pen	Pen
Pen	Paper Sheet
Pencil	Pencil
Pencil	Sharpener
Paper Sheet	Paper Sheet
Paper Sheet	Pen
Sharpener	Sharpener
Sharpener	Pencil

You may observe that the output of the command has many extra records like records with the same item, e.g. the pair (Pen, Pen). In addition, a pair like (Pen, Paper Sheet) is same as (Paper Sheet, Pen), therefore, only one of these pairs should appear in the result. A simple solution to this problem is to add another condition in the SQL command: (i1.ItemID < i2.ItemID). This will ensure that only those pairs in which ItemID of first table is less than ItemID of second table will be part of output, thus, eliminating both the problems, as stated above. The command is as follows:

```
SELECT i1.ItemName, i2.ItemName
FROM ITEM i1, ITEM i2
WHERE i1.ItemPrice = i2.ItemPrice AND i1.ItemID < i2.ItemID;
```

Thus, you are joining the two tables on identical ItemPrice, and displaying the pair of names of the items that have similar prices. You will get the following output of this SQL command:

i1.ItemName	i2.ItemName
Pen	Paper Sheet
Pencil	Sharpener

Check Your Progress 1

Consider the Tables, given in Figure 1, and answer the following questions:

- 1) (a) Find the list of those item names that have been supplied, as part of at least one Order.
 (b) List the order details including the order id, item code, item name and price.
 (c) Compute the total price of an order.

.....

.....

.....

.....

- 2) Find the list of all the item IDs, item names and related Order IDs. This query should list the name of the items even if it is not part of any order.

.....

.....

.....

- 3) Find the list of items that have been purchased together.

.....
.....
.....

8.3 NESTED QUERIES

In the previous section, we discussed join queries, which are responsible for joining the data from two tables. Nested queries are used when the output of a query may be useful for the execution of another query. Thus, you can create a main query nest a sub-query in any of the clauses of this main query. In this section, we discuss the basic type of nested queries and then a special type of nested subqueries called correlated subqueries.

8.3.1 Sub-queries

A sub-query is another SELECT statement that is used in the main SELECT statement. Please remember the following points about a subquery:

- A sub-query is executed prior to the main query. Therefore, you can use the result of a sub-query in an expression of the main query.
- A sub-query, on its execution, can return either a single value or a set of values or a relation. Therefore, the result of the sub-query can be used in a comparison in the WHERE or HAVING clause or for comparison with a set operator; or even in the FROM clause when a relation is returned.
- You can put a sub-query inside a sub-query. You can use a separate table in the main and sub-query.
- You should not use the ORDER BY clause in a sub-query, rather it should be used as the last clause of the main query. However, you can use the GROUP BY clause in the sub-query.
- When you use sub-queries in the WHERE or HAVING clause of the main query, you may be required to use comparison or set operators. Some of these operators are given below:

Comparison Operators	<, =, >=, <, <=, <>	Used when the sub-query returns a single constant value
Set Operators	IN, ANY, ALL	Used when sub-query returns a set of values.

These operators will be explained in various examples. We will use tables and data from Figure 1 to answer the following queries.

Example 4: Find the item, whose available quantity is the smallest.

```
SELECT      *
FROM        ITEM
WHERE       QuantityAvailable = ( SELECT      MIN (QuantityAvailable)
                                FROM          ITEM);
```

The sub-query in this case will find a single minimum value, which will be compared in WHERE clause to determine the record, which has that minimum value.

ItemID	ItemName	ItemPrice	QuantityAvailable
I04	Sharpener	5	200

Example 5: Find the item, whose price is the minimum.

```
SELECT      *
FROM        ITEM
WHERE       ItemPrice = (      SELECT      MIN (ItemPrice)
                              FROM        ITEM);
```

The sub-query in this case has found a single minimum value, which will be compared in the WHERE clause. Please note that in this case, the output contains two records.

ItemID	ItemName	ItemPrice	QuantityAvailable
I02	Pencil	5	1000
I04	Sharpener	5	200

Example 6: Find the items, whose price is greater than the price of a Pencil and the quantity available is more than the Item whose ItemID is I01. Assume that item names are unique.

```
SELECT *
FROM ITEM
WHERE ItemPrice > ( SELECT ItemPrice
FROM ITEM
WHERE ItemName = "Pencil")
AND QuantityAvailable > ( SELECT QuantityAvailable
FROM ITEM
WHERE ItemID = "I01");
```

There are two sub-queries in this case, the output of the first sub-query would be value 5 and the output of the second sub-query would be value 500. The result of the query is given below:

ItemID	ItemName	ItemPrice	QuantityAvailable
I03	Paper Sheet	20	1000

Example 7 (a): Find the total quantity of all the items in every order.

This query can be answered by the ORDERDETAILS table using a simple GROUP BY clause:

```
SELECT OrderID, SUM(QuantityPurchased) AS TotalofAllItems
FROM ORDERDETAILS
GROUP BY OrderID;
```

OrderID	TotalofAllItems
O001	13
O002	5
O003	7
O004	8
O005	8

Example 7 (b): Find the total quantity of all the items for the orders, which have ordered more total quantity than ordered in order O003.

[illegible]

Please note that the sub-query in this case is in the HAVING clause. This command will output:

OrderID	TotalofAllItems
O001	13
O004	8
O005	8

Example 8(a): Find the total price of each Order.

This query would require joining of ORDERDETAILS table and ITEM table, as the details of an order are in the ORDERDETAILS and Price of each order is in ITEM table. The following command would simply compute the Price of each Item of the order:

```
SELECT ORDERDETAILS.*, ItemName, ItemPrice, (QuantityPurchased*Price) AS ItemTotal
FROM ORDERDETAILS, ITEM
WHERE ORDERDETAILS.ItemID = ITEM.ItemID;
```

This command will list the orders as under:

OrderID	ORDERDETAILS.ItemID	QuantityPurchased	ItemName	ItemPrice	ItemTotal
O001	I02	2	Pencil	5	10
O001	I03	10	Paper Sheet	20	200
O001	I04	1	Sharpener	5	5
O002	I02	5	Pencil	5	25
O003	I01	2	Pen	20	40
O003	I03	5	Paper Sheet	20	100
O004	I01	5	Pen	20	100
O004	I03	3	Paper Sheet	20	60
O005	I01	8	Pen	20	160

To find the answer of the stated query, you need to find the sum of item total for each order. Thus, you can answer of the stated query by the following SQL command:

```
SELECT OrderID, SUM(QuantityPurchased*Price) AS OrderTotal
FROM ORDERDETAILS, ITEM
WHERE ORDERDETAILS.ItemID = ITEM.ItemID
GROUP BY OrderID;
```

This command will list the orders as under:

OrderID	OrderTotal
O001	215
O002	25
O003	140
O004	160
O005	160

Example 8(b): Find the total price of the orders, which have at least one order for the Item whose item name is Paper Sheet.

This query would require you to use the answer of the query 8(a) and use of join in the subquery. The answer to the subquery would be a set of records, therefore, we will use set operator IN. The query is given as follows:

```

SELECT OrderID, SUM(QuantityPurchased*Price) AS OrderTotal
FROM ORDERDETAILS, ITEM
WHERE ORDERDETAILS.ItemID = ITEM.ItemID AND OrderID IN
    ( SELECT DISTINCT OrderID
      FROM ORDERDETAILS oo, ITEM i
      WHERE oo.ItemID = i.ItemID AND
            ItemName = "Paper Sheet")

GROUP BY OrderID;

```

This command will list the orders as under:

OrderID	OrderTotal
O001	215
O003	140
O004	160

Example 9: Find the id and name of all the clients, who have ordered Pen and Paper Sheet in a single order. This query would require the joining of the ORDER table and CLIENT table in the main query and the ORDERDETAILS table and ITEM table in the sub-query. The following command would find the list of OrderIDs, which have both Pen and Paper Sheet in a single Order.

```

(SELECT DISTINCT OrderID
 FROM ORDERDETAILS, ITEM
 WHERE ORDERDETAILS.ItemID = ITEM.ItemID AND ItemName= "Pen")
INTERSECT
( SELECT DISTINCT OrderID
 FROM ORDERDETAILS, ITEM
 WHERE ORDERDETAILS.ItemID = ITEM.ItemID AND ItemName= "Paper Sheet");

```

This query will result in the following output:

OrderID
O003
O004

Now, you can write the query as follows:

```

SELECT DISTINCT (CLIENT.ClientID, ClientName)
FROM ORDER, CLIENT
WHERE ORDER.ClientID = CLIENT.ClientID AND
      OrderID IN (
        (SELECT DISTINCT OrderID
         FROM ORDERDETAILS, ITEM
         WHERE ORDERDETAILS.ItemID = ITEM.ItemID AND ItemName= "Pen")
        INTERSECT
        ( SELECT DISTINCT OrderID
          FROM ORDERDETAILS, ITEM
          WHERE ORDERDETAILS.ItemID = ITEM.ItemID AND ItemName= "Paper Sheet")
      );

```

This command will list the client's information as under:

<u>ClientID</u>	ClientName
C001	ABCD
C003	CDEF

Example 10: List the total price of the orders, whose total price is more than 150.

An alternate way of writing this query would be to use a subquery in the FROM clause, as given below:

```
SELECT OrderID, OrderTotal
FROM (SELECT      OrderID, SUM(QuantityPurchased*Price) AS OrderTotal
      FROM        ORDERDETAILS, ITEM
      WHERE        ORDERDETAILS.ItemID = ITEM.ItemID
      GROUP BY OrderID)
WHERE OrderTotal > 150;
```

<u>OrderID</u>	OrderTotal
O001	215
O004	160

You may please note that an alias in the subquery is used in various clauses of the main query.

8.3.2 Correlated Subqueries

In many queries, the columns of the tables used in FROM clause are also used in the subquery. Such subqueries are called Correlated subqueries and generally are time-consuming queries. These queries are explained with the help of the following example.

Example 11: List the client id and name of the clients, who have ordered ItemID I01 and ItemID I03, as part of a single order.

This query can be answered as a correlated query as follows:

```
SELECT DISTINCT OrderID
FROM ORDERDETAILS outer
WHERE ORDERDETAILS.ItemID = "I01" AND EXISTS
( SELECT DISTINCT OrderID
  FROM ORDERDETAILS inner
  WHERE outer.OrderID=inner.OrderID AND
        outer.ItemID<inner.ItemID AND inner.ItemID = "I03"
);
```

The execution of such a correlated query is time-consuming. Why? Since in these queries, the subquery is executed for each instance of the main query. For example, in the case of example 11, the main query will find that the order O003 fulfils the main clause. This will trigger the execution of the subquery, which will check that for the same value of order O003, there exists a record for item I03. Since it exists, therefore O003 will be output. Similarly, O004 will be output. In the case of O005, the main query has the record for I01, however, the record for I03 does not exist. Thus, O005 will not be output. Thus, this query will result in the following output:

<u>OrderID</u>
O003
O004

Check Your Progress 2

- 1) What is a subquery? When would you like to use the sub-query?

.....

.....

.....

- 2) Consider the following relations:

EMPLOYEE (EmployeeNo, EmployeeName, EmployeePhone, EmployeeBasicPay)

PROJECT(ProjectNo, ProjectName, ProjectDuration)

EMPLOYEEPROJECT (EmployeeNo, ProjectNo, EmpRoleInProject)

Now answer the following queries:

- (a) Find the name of the projects, which have the least duration.
(b) Find the name and basic pay of employees who are working on more than one project.
(c) Find the name of the employee who earns more than the average salary of all the employees.

.....

.....

.....

- 3) Consider the following relations Schema given in question 2 above, and the following two SQL queries on this schema. What is the purpose of these two queries?

(i) SELECT EmployeeName
 FROM EMPLOYEE
 WHERE EmployeeNo IN (SELECT EmployeeNo
 FROM EMPLOYEEPROJECT
 GROUP BY EmployeeNo
 HAVING COUNT(EmployeeNo) =
 (SELECT COUNT (*) FROM PROJECT));

(ii) SELECT ProjectName
 FROM PROJECT
 WHERE ProjectNo IN ((SELECT ProjectNo
 FROM PROJECT)
 MINUS
 (SELECT DISTINCT ProjectNo
 FROM EMPLOYEEPROJECT)
);

8.4 DATABASE OBJECTS

Database objects are useful concepts in a database system. In this section, we discuss four different types of objects, which are defined in many database management systems. Views are virtual tables, which may be used for implementing database security. In addition, they can also be used for database query optimisation. Sequences are used to maintain an automatic sequence of numbers, which can be very useful for input of unique values in a column. Indexes are used to enhance the performance of a database system. The following sub-section discusses these concepts in detail.

8.4.1 Views

A view is a part of external schema of a database, which can be used to restrict the data display, input and modification of a database system. A database system consists of base tables, which are the tables created for an entity or relationship of a logical or conceptual database model. The base tables are used to store data. A view can be categorised as a virtual table, which can be stored as a query on a table. This query is called the view definition. Please note that views do not store data of their own, rather data is stored in the base tables of the database. The query definition of a view can be stored in the data dictionary of DBMS. When you use a view, then it can be executed through the following steps:

- Retrieve the view definition from the data dictionary.
- Find the access privileges of the view, as well as the base tables associated with the view.
- Create the view qualified query, if the user has access privileges on the view and data of base tables.

The following are the Advantages of using views in a database system:

- Views allow database users to access only that data to which they have access privileges.
- Views mechanisms enforce logical data independence in a database system.
- Views allow different users to view the data differently.

How to create a view?

A view is created using a CREATE VIEW statement followed by a query. A view statement can include multiple tables in the FROM clause, the condition of joining two tables in the WHERE clause, the GROUP BY clause and expressions of a subquery. However, a view statement is not allowed to include an ORDER BY clause. The following are examples of view creation.

Example 12: Create a view for the logistics person, who would be interested in the list of all the items (with the item name and quantity) of a particular order, say O001, and the address of the client to whom that order is to be sent.

```
CREATE VIEW LOGISTICS AS
```

```

SELECT oo.OrderID OOID, oo.ItemID IItemID, ItemName, QunatityPurchased,
       c.ClientID CClientID, ClientName, ClientAddress, ClientPhone
FROM ORDERDETAILS oo, ITEM i, ORDER o, Client c
WHERE oo.ItemID = i.ItemID AND oo.OrderID = o.OrderID
      AND o.ClientID=c.ClientID AND oo.OrderID= "O001";

```

This will create a view named LOGISTICS. you can display the content of the LOGISTICS view using the following command:

```

SELECT *
FROM LOGISTICS;

```

It will display the following output:

OOID	IItemID	ItemName	QuantityPurchased	CClientID	ClientName	ClientAddress	ClientPhone
O001	I02	Pencil	2	C001	ABCD	79, MGRoad	9999999991
O001	I03	Paper Sheet	10	C001	ABCD	79, MGRoad	9999999991
O001	I04	Sharpener	1	C001	ABCD	79, MGRoad	9999999991

Data Updates through Views?

Views can be used for data updates though there are several restrictions on updating data through the views. In general, you cannot update the data through a view, if it contains a JOIN operation or a GROUP BY clause or an aggregate function or a subquery. In addition, you may use a check option for views, which is defined next.

Creating views with check option: Consider a view that allows updates on data, but those updates may result in a modified record, which does not fulfil the view defining criteria. The following example explains this situation.

Example 13: Create a view of the items whose price is more than INR 15.

```

CREATE VIEW COSTLYITEM AS
SELECT *
FROM ITEM
WHERE ItemPrice > 15;

```

This will create the view consisting of two items I01 and I03. Assume that the price of the Pen is discounted and made Rs 10, if you allow an update through the COSTLYITEM view, then the price of item I01 (Pen) will be updated to INR 10. Thus, I01 will no longer be part of the view and disappear from the view.

The situation may be avoided if you use WITH CHECK OPTION, which will restrict such updates. Thus, you may use the following option to create the view:

```

CREATE VIEW COSTLYITEM AS
SELECT *
FROM ITEM
WHERE ItemPrice > 15
WITH CHECK OPTION;

```

8.4.2 Sequences

Consider a situation, where in a column, you want to assign the next number in a sequence in every new record. Sequences are used for generating a new unique number of a sequence for a specific column. Sequences may be used for automatically generating the unique primary key values, as they can be generated efficiently. The following example explains the use of sequences:

Example 14: You can create a sequence, say ENRSEQ, for generating unique enrolment numbers of the students as follows (Please note different DBMS may have a different command for such sequences):

```
CREATE SEQUENCE ENRSEQ
START WITH 230000001
INCREMENT BY 1
MAX VALUE 239999999;
```

The SQL commands, as given in example 14 will generate the sequence numbers ENRSEQ, but how will you use these sequences in tables? This is explained in example 15.

Example 15: This example demonstrates the use of sequence numbers while inserting records in a table. Consider a relation STUDENT (StudentID, StudentName, StudentProgCode). The StudentID is the primary key of the relation and should be unique and 9 characters long in the range 230000001 to 239999999. To automatically generate the sequence of student ids you may use the following command:

```
INSERT INTO STUDENT VALUES (ENRSEQ.NEXTVAL, "ABCD", "PGDCA");
INSERT INTO STUDENT VALUES (ENRSEQ.NEXTVAL, "BCDE", "BCA");
INSERT INTO STUDENT VALUES (ENRSEQ.NEXTVAL, "CDEF", "PGDCA");
```

You can display the content of the table after these insertions using the following commands:

```
SELECT * FROM STUDENT;
```

<u>StudentID</u>	StudentName	StudentProgCode
230000001	ABCD	PGDCA
230000002	BCDE	BCA
230000003	CDEF	PGDCA

You may please note that all the details of a sequence, like starting number, increment value and maximum values are stored and updated from time to time in the data dictionary. Further, certain situations like the rollback of an insert operation may result in gaps in the inserted values of a sequence.

Can you modify a sequence, once created? Yes, for this purpose, you may use the ALTER SEQUENCE statement of SQL. For example, you can change the increment value to 5 and the maximum value to 235000000 for the sequence ENRSEQ using the following command: the

```
ALTER SEQUENCE ENRSEQ
START WITH 230000001
```

```
INCREMENT 5  
MAX VALUE 235000000;
```

You can also delete a sequence by giving the following command:

```
DROP SEQUENCE ENRSEQ;
```

8.4.3 Indexes and Synonyms

An index is part of the physical/internal schema of relational design. It is one of the important access structures of a database. Indexes help in enhancing the speed of the query processing. Every database has a basic set of indexes, like the primary key index, which are created automatically by the database management system. Additional indexes can be created by the database administrator, if needed.

You can create indexes using the following command:

Example 16: Consider the STUDENT relation, as given in Example 15. You need to create indexes for (i) To enhance the queries related to various programmes (ii) to list the students of a programme in the order of their names, for this case, you may like to create an index on programme and student name. You can use the following commands to create these indexes:

```
CREATE INDEX PROGCODE_IDX ON STUDENT (StudentProgCode);  
CREATE INDEX PROG_STUDENT_IDX ON STUDENT (StudentProgCode, StudentName);
```

It may be noted that the information of the indexes of the tables is stored in the data dictionary of a DBMS.

Further, you may note that though an index may help in improving query response time, they increase the time of data insertion, modification, and deletion, as these operations require updating the contents of related indexes. In general, you cannot modify the structure of an index, however, you can delete an index. To remove an index, you may give the command:

```
DROP INDEX PROGCODE_IDX;
```

Several DBMSs allow creation of alternative links or names to the database items or objects. These alternative names are generally short names and used for convenient references. For example, a typical DBMS may allow the creation of short names using the following command:

Example 17: You may create the following synonym for the STUDENT tables of example 15.

```
CREATE SYNONYM st  
FOR STUDENT;
```

You can remove a synonym by giving the following command:

```
DROP SYNONYM st;
```

Check Your Progress 3

1) Consider the following relations:

```
EMPLOYEE (EmployeeNo, EmployeeName, EmployeePhone, EmployeeBasicPay)  
PROJECT(ProjectNo, ProjectName, ProjectDuration)  
EMPLOYEEPROJECT (EmployeeNo, ProjectNo, EmpRoleInProject)
```

Create a detailed view of Projects consisting of Project number, project name, employee name of employees working on the project and his/her role in the project.

.....
.....
.....

- 2) Create a three-digit long odd number sequence.

.....
.....

- 3) What are the advantages of creating indexes?

.....
.....

8.5 SUMMARY

This unit introduces you to join queries and sub-queries. First, the unit explains the use of the JOIN statement in SQL. A JOIN statement uses two tables each of which has at least one attribute on the same domain. These attributes also called joining attributes, are used to join the tables using a conditional operator. In case, this conditional operator is equality, then the join operation is called the equi-join. In addition, if the joining attributes have the same name in both the tables, then only one of these attributes is included in the output. This type of join is called the Natural join. Outer joins are used when the records of the table, which do not participate in join operation are to be output too. Self-join is performed when one column values are to be related to the same or another column of the same table. The joining condition of self-join requires removing redundant records. The unit discusses these joins with the help of examples. Next, the unit discusses the nested queries. The unit details some of the simple operators that are used with sub-queries with the help of examples. This unit also discusses the correlated subqueries, which are generally expensive to execute. Finally, the unit provides details of some of the important concepts used in database management systems such as views, sequences and indexes.

You should practice these queries using a DBMS.

8.6 SOLUTIONS/ANSWERS

Check Your Progress 1

- 1) (a) The information about what parts have been supplied as a part of at least one order is available in the ORDERDETAILS table, it may be joined with the ITEM table to find the names of the parts.

```
SELECT DISTINCT ITEM.ItemID, ItemName
FROM ITEM, ORDERDETAILS
WHERE ITEM.ItemID = ORDERDETAILS.ItemID;
```

You can also use a subquery (explained in the next section) for this information:

```
SELECT ItemID, ItemName
FROM ITEM
WHERE ItemID IN ( SELECT DISTINCT ItemID
                  FROM ORDERDETAIL
                );
```

(b) The information about what orders are available is in the ORDERDETAILS table and information about the items is available in the ITEM table. Therefore, you need to join these two tables for the required information.

```
SELECT OrderID, ORDERDETAILS.ItemID, ItemName, ItemPrice
FROM ORDERDETAILS, ITEM
WHERE ORDERDETAILS.ItemID = ITEM. ItemID;
```

(c) The following query may result in the desired information:

```
SELECT OrderID, SUM(QuantityPurchased*ItemPrice)
FROM ORDERDETAILS, ITEM
WHERE ORDERDETAILS.ItemID = ITEM. ItemID
GROUP BY OrderID;
```

2) It will require outer join:

```
SELECT ITEM.ItemID, ItemName, OrderID
FROM ITEM LEFT OUTER JOIN ORDERDETAILS
ON ITEM.ItemID = ORDERDETAILS.ItemID;
```

3) This will require a self-join.

```
SELECT od1.ItemID, od2.ItemID
FROM ORDERDETAILS od1, ORDERDETAILS od2
WHERE od1.OrderID = od2.OrderID AND od1.ItemID < od2.ItemID;
```

Check Your Progress 2

- 1) A subquery is a SELECT statement that is part of a main SELECT statement. You may put the SELECT statement in the FROM, WHERE or HAVING clauses of the main SELECT statement. Subqueries are useful when you want to use the set operators. They can also be used where a smaller relation can be returned and used more efficiently.
- 2) (a) Find the name of the projects, which have the least duration. (Can you write a more efficient query in this case?)

```
SELECT ProjectName
FROM PROJECT
WHERE ProjectNo IN
      (SELECT MIN(ProjectDuration)
       FROM PROJECT );
```

(b) Find the name and basic pay of employees who are working on more than one project.

```
SELECT EmployeeNo, EmployeeName, EmployeeBasicPay
FROM EMPLOYEE
```

```
WHERE EmployeeNo IN
      (SELECT EmployeeNo)
FROM EMPLOYEEPROJECT
GROUP BY EmployeeNo
HAVING Count(ProjectNO) > 2);
```

(c) Find the name of the employee who earns more than the average salary of all the employees.

```
SELECT EmployeeNo, EmployeeName, EmployeeBasicPay
FROM EMPLOYEE
WHERE EmployeeBasicPay >
      (SELECT avg (e.EmployeeBasicPay)
FROM EMPLOYEE e);
```

- 3) (i) To find names of employees who are working on all projects.
(ii) To find names of the projects on which no employee is presently working.

Check Your Progress 3

1.

```
CREATE VIEW PROJECTDETAILS AS
SELECT p.ProjectNo, ProjectName, EmployeeName, EmpRoleInProject
FROM EMPLOYEE e, EMPLOYEEPROJECT ep, PROJECT p
WHERE e.EmployeeNo = ep.EmployeeNo AND ep.ProjectNo = p.ProjectNo;
```

2.

```
CREATE SEQUENCE ODDSEQ3
START WITH 101
INCREMENT BY 2
MAX VALUE 999;
```

3.

Indexes are very useful for answering queries using non-key attributes. However, they enhance overheads in terms of maintaining indexes.